

DATA CHALLENGE DEEPLARNING

FINAL REPORT

Classifying X's users as influencers or observers

December 1, 2025

MP3,5 – Roman Lendormy, Arthur Buis
roman.lendormy@polytechnique.edu
arthur.buis@polytechnique.edu



1. DATA PROCESSING

For this data challenge, we had to predict user classes (either influencer (=1) or observer (=0)) based on a training dataset of more than 154,914 tweets in JSON format. To validate our model, we had a set of 103,380 untagged tweets. The training dataset was dense (192 columns, excluding labels) and therefore required rigorous cleaning. First, we eliminated 48 constant (and therefore useless) columns and for many features, we aggregated a first field with a secondary field contained in `extended_tweet` (for example to obtain `full_text`).

The most significant features for our models are those carried by the user field. However, the number of followers (`user.follower_count`), which could have been the most discriminating data for our classification, has been removed from the dataset. We selected the fields `user.favorites_count` (number of favorites for the account) (Fig 1a), `user.listed_count` (number of public lists in which the user is included) (Fig 1b), and `user.statuses_count` (number of posts by the user) (Fig 1c): in fact, we can observe two distinct distributions of features based on the labels, which makes them interesting as discrimination factors. Furthermore, as these distributions occur over very different numerical ranges, we chose to use the logarithm of these features in our models, as this seemed more relevant for transcribing the characteristics of the set. We also selected binary data such as `user.geo_enabled`, `user.profile_use_background_image`, `user.default_profile`, and `user.profile_background_tile`, which are sufficiently heterogeneous in terms of the values taken from the dataset and rather discriminating for our two classes (Fig 2). Finally, we created the field `all_user_mentions_count`, which counts the number of users identified in each tweet and shows a slight correlation between the number of mentions and belonging to class 0.

The `quoted_status` field, relating to the tweet to which the user replied, was rather disappointing after analysis: it only showed a weak correlation with the labels. We nevertheless chose to use `quoted_status.favorite_count` (number of times the tweet has been added to favorites) and `quoted_status.user.favorites_count` (the number of favorites of the user who posted) for our model in order to diversify the data and because they seemed relevant to us. As with the numerical fields for users, we chose to apply a logarithm to these features. Another field that interested us greatly was `source`, with the idea, for example, that influencers might tweet from specific interfaces. Looking at the data, we can see a few very dominant sources (iPhone, Android, iPad), sources from tweet schedulers (Tweetdeck, Hootsuite), and a few very minor sources, but often specific to a category of users. We therefore carried out partial one-hot encoding: we fully encoded the most important sources, and for the remaining ones, we created the field `other_1` which identifies other sources appearing more than three times, all of which are labeled 1, and `other_0` which does the same thing with the label 0.

Finally, we took into account the data in entities, and in particular `entities.media` and `entities.hashtags`, which seem to be good discriminators for our model (Fig 3). For `entities.media`, we created one-hot `has_photo`, `has_video`, and `has_gif` depending on the type of media contained in the tweet, and for entities hashtags, we identified the 20 most frequently used hashtags in the set and used one hot encoding when a tweet contained them. We tried many other features, such as the ratio of `user.favourite_count` to `user.listed_count`, but after analyzing our models with IntegratedGradients (which varies the value of certain features to see the impact on predictions) (Fig 5), we eliminated them because they were either redundant or had no effect. This allowed us, for example, to eliminate many ineffective hashtag encodings. Finally, we chose to use text in our models, passing it through different encodings. First, the full text, merging the text of the first field with that of `extended_tweet`, but also the text in `user.description`, the user's account description.

Since the number of followers seems to be one of the most promising features, we tried to reconstruct it, at least partially. We found a way to identify users in the dataset using the precise date and time (to the millisecond) when their account was created (`user.created_at`), which is a relatively robust key for identification. After studying it, we found that there are very few collisions in the training set (less than 1%) and no collisions with the label field. Since the number of followers is given in the `quoted_status` field, we attempted to match `quoted_status.user.created_at` with `user.created_at`. However, this matching was quite limited and we

did not use it (it only works for 1% of the dataset), but with this data we observed a clear correlation between the number of followers and the label. Nonetheless, grouping different tweets with the same `user.created_at` allows us to identify tweets from the same users. We used this identification at the output of our models to average the predictions of tweets from the same users, slightly improving (0.05% with Bertweet) the final result obtained.

In this challenge, we sought to understand the complex models deployed, and in particular how textual data is encoded. We were therefore able to perform PCA and T-SNE on the embeddings obtained with Bertweet after training our second transformer model. Executed on the one hand with `user.description` and on the other hand with `full_text`, we obtained very different results for this model, which is supposed to transcribe tweets accurately. In the following figures (Fig 4), the conclusion is clear: the model represents `user.description` very well but fails on `full_text`, which is why we preferred the first field to the second in our textual models.

2. BASIC MODELS

The first approach proposed for the challenge was based on TF-IDF encoding of the full text contained in the tweets in the form of unigrams or bigrams, followed by logistic regression. After training managed by K-Fold, we can see an average accuracy of 62.59%, which after evaluation on Kaggle gives 53.8%, highlighting an overfit on the lexical features given by TF-IDF.

A second basic approach we propose is also a logistic regression, but this time on the PCAs of text embeddings after training a transformer-type model. To do this, we train a simple transformer model consisting of a first fine-tuning layer of the Bertweet model applied to our text data, followed by a Dropout layer and then a linear layer for classification. We then use the learned encoding as a new representation for the text of the tweets, to which we apply a 50-component PCA. We also tested a 2-component dimension reduction with t-SNE based on the PCA result, with 2,000 iterations and a perplexity of 30. In the figures obtained (Fig 4), we observe a clear separation according to class, particularly for `user.description`, which was therefore our preferred choice for textual features. As the separation was quite clear, we tried to see what a logistic regression on the PCA would give. For 2,000 iterations from the text field `full_text`, we obtained an accuracy of 0.73 compared to 0.92 with the field `user.description`, which corresponds to obvious overfitting: however, we obtained a score of 66.8% from the regression on the description with the Kaggle data, which is an improvement on the previous model.

As a first approach, we also studied the performance of a CNN based entirely on text. The tokenizer used at that time was the Keras Tokenizer, which simply takes into account word frequency. From these tokens, the CNN produces 128-dimensional embeddings, then a 1D convolution is applied in order to take into account the local order of words and their position in the text. The model then consists of a high dropout layer (0.5) and a final linear layer for classification. The model overfits very quickly, so three epochs are sufficient to achieve optimal accuracy on validation, which is 0.65. On Kaggle data, the score is also in this range (63.8%) for 1 min 30 s of training. We could have tried to use more sophisticated tokenizers, but we preferred to turn to transformer models for this purpose.

3. TRANSFORMER MODELS

The first transformer model we considered is fairly simple: it consists of a layer that produces embeddings from the text, a second classification layer that retrieves the metadata and embeddings and creates a new representation, and finally a linear layer for classification. We chose manual tokenization as our initial approach, counting each word or emoji as a token and keeping only the most frequent ones (we obtained fairly comparable

scores using `full_text` or `user.description`). For the transform layer, we used the embedding already present in PyTorch, defined a Positional Encoding layer, and finally used PyTorch's encoder layers. However, this model was very limited: beyond two epochs, the model overfit significantly, even with very low learning rates, and the accuracy was very sensitive to hyperparameters. We attributed this to the size of the training data: 150,000 tweets are not enough to compose a representative vocabulary, leading the model to either overfit or under-represent the text in the validation sets. However, since training this model takes 5 minutes on the school's GPUs, we still performed a grid search to optimize the parameters for different values of the learning rate, hidden layer dimensions, attention heads, linear layers, and epochs, which ultimately allowed us to achieve a score of 79.3% on Kaggle (79.5% validation accuracy) by gradually adjusting the vocabulary size.

It was the text processing part that caused us the most problems in the previous phase, so we considered pre-trained models to improve our embeddings. We tried Bert, Bertweet, and finally CamemBert, which is more suited to French texts. The overall architecture of the model is the same as in the previous model, but the intermediate layers are more complex: a text module (camemBert) produces embeddings, a subnetwork with two linear layers and a Dropout layer produces encodings on the numerical features, then the two outputs are concatenated before being fed into a final three-layer classifier. As the two sub-layers are very different and the number of parameters is very high (111 M), we chose to break down the model training into two phases: in the first phase, Bert's parameters are completely frozen in order to train only the metadata layer and the final linear layer (warm-up phase, the model learns BERT embeddings without modifying BERT) with a scheduler to gradually adapt the learning rate. In the second phase, the last two layers of BERT are unfrozen in order to fine-tune it to the data, with different learning rates per layer group (lower for the classification layer, for example, so as not to completely cancel out the learning from phase 1). This architecture proved promising, allowing us to achieve 83.4% on the Kaggle dataset with Bertweet, then 84.0% with Camembert, after approximately 40 minutes of training. However, training in phase 2 often caused us problems with overfitting for more than one epoch (Fig 6). For a randomly selected training and validation set, the model overfit the validation set (it was able to make matches on metadata in particular), and when we built the sets so that there were no common users between the two, we ended up with overfitting of the training set and stagnant validation accuracy. Despite attempts to fine-tune the learning rate, exceeding one epoch caused our Kaggle score to drop.

Finally, we looked for a way to exploit both `full_text` and `user.description` at the same time, and we turned to a dualbert model that takes two text modules instead of just one. The textual data is encoded with camemBert, a first text module produces embeddings of size 768 for `user.description` and a second separate one for `full_text`, followed by two linear layers and a dropout layer for metadata training, and finally a final classification layer composed of three linear layers that provides a prediction based on the stacked data. This time, training is broken down into three phases: the first for the metadata layer and classification, with frozen BERTs; a second in which the BERT for `user.description` is unfrozen; and finally, in the third phase, the parameters of the last BERTs are finally modifiable. With 222M parameters, this model overfit reasonably on the last training phase despite learning rates adapted to each phase. This model allowed us to achieve up to 83.9% on the Kaggle dataset. However, since the score is not really improving and the training time is around 3 hours, we have chosen to keep the previous model with only `user.description` for our final prediction.

4. XGBOOST

In parallel with the experiments described above, we tried different training configurations with the XGBoost model. XGBoost is a decision tree-based boosting algorithm that combines several weak models iteratively to correct the errors of the previous ones. It is known for its robustness, its ability to handle heterogeneous features, and its excellent compromise between performance and computation time. We felt it had the best potential for success in our task, as our dataset contains a mix of numerical, categorical, and textual data, and XGBoost is particularly effective at capturing these types of complex relationships.

First, to familiarize ourselves with the model and get to grips with it in a simplified context, we chose to

run it on an extremely simplified version of the dataset, which contained only a few features deemed relevant, such as the text of the tweet, the date the account was created, and the user's description. To vectorize the textual data, we chose to use TF-IDF, for the same reason of simplicity. This approach, combined with a search for the most effective hyperparameters, enabled us to obtain a score of up to 0.785. For the hyperparameter search, we chose to implement a grid search: costly, but acceptable given the small size of the training dataset.

We then moved on to the most complete version of the dataset presented in the first part, initially without including the textual data. At first, we did not exceed a score of 0.83, which motivated us to add the textual data with a more sophisticated vectorization than the TF-IDF used previously. We chose to use the BERT and then BERTweet models for this task, which, to our surprise, did not improve the score. After reflection, we realized that these models are adapted for the English language, and we decided to turn to CamemBERT, which allowed us to gain a few additional thousandths. The text of the tweets and the user description are therefore vectorized in spaces of dimension 768.

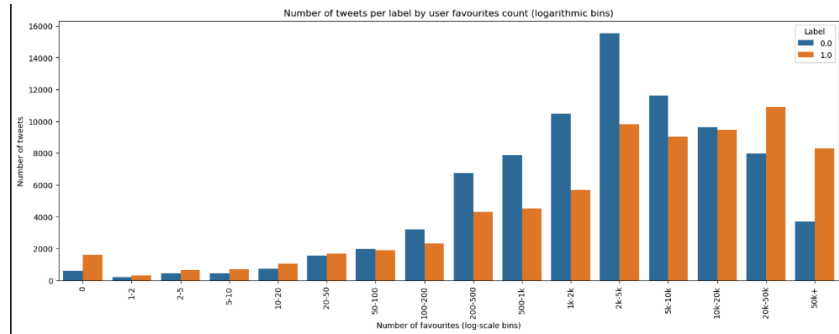
However, an overfitting problem remained: our Kaggle score was consistently lower than our score on the evaluation set. We eventually identified a possible source of error: some users had part of their tweets in the training set and another part in the validation set. Since users in the training set were probably not in the Kaggle evaluation data, this explained the difference in scores. We managed to solve this problem by separating the training set and the evaluation set by distinct users. Unfortunately, this change did not improve our Kaggle scores, so we decided to keep the version with overfitting as the final version of xgboost, even though we recognize that it is not optimal.

In order to better estimate the generalization ability of our model and reduce overfitting, we implemented k-fold cross-validation. This method consists of dividing the training dataset into k subsets (folds) of similar size, then training the model k times: at each iteration, one fold is used as a validation set while the remaining k-1 folds are used for training. The final evaluation is obtained by averaging the performance across all folds, which limits the risk of the model overfitting to a specific portion of the data. Despite this robust approach, we did not observe any significant improvement in our Kaggle score. For k=5, model accuracy ranged between 0.83 and 0.84, but the overall score did not rise above 0.83.

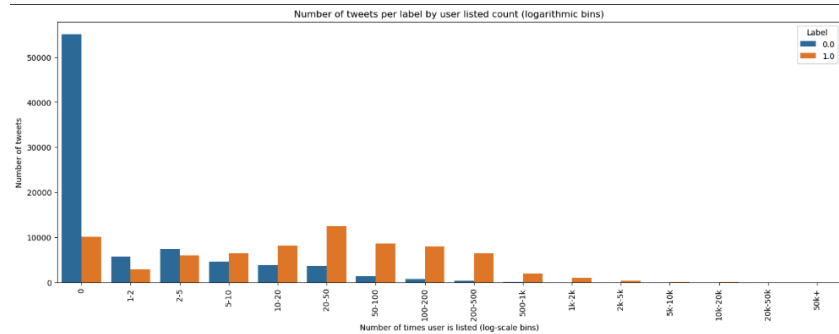
5. WEIGHT AVERAGING AND SOFTENING

Finally, we chose to average the probabilities of the two final models (Transformer with camemBert and XGBoost) to obtain the best possible prediction. Once the final probabilities were retrieved, we averaged the tweets by user: we identified them with `user.created_at` and calculated an average per user. This average allowed us to raise the score by a few points, ultimately achieving a final score of 84.2% on Kaggle.

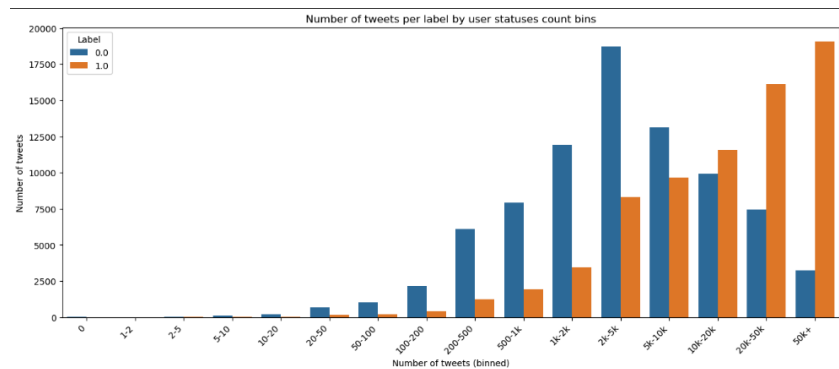
ANNEX & FIGURES



(a) Favourites count per label

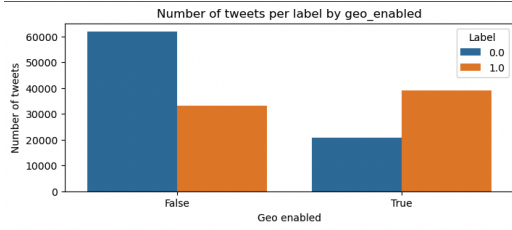


(b) Listed count per label

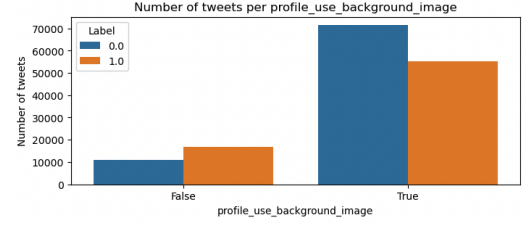


(c) Statuses count per label

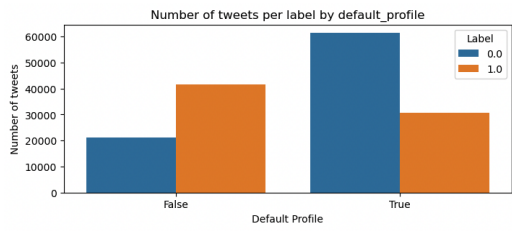
Figure 1: User fields per label



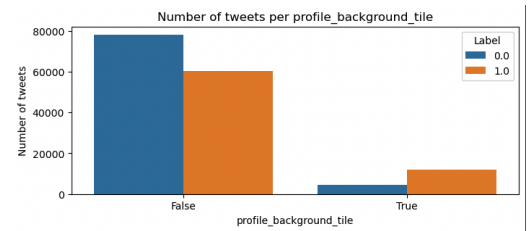
(a) Values of geo_enabled per label



(b) Values of profile_use.background.image per label

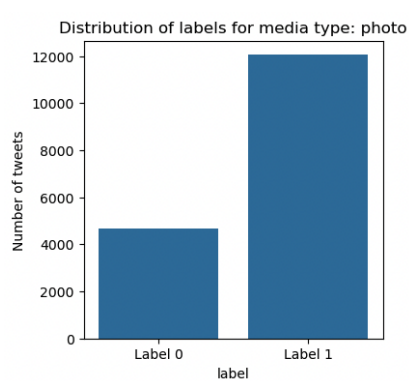


(c) Values of default_profile per label

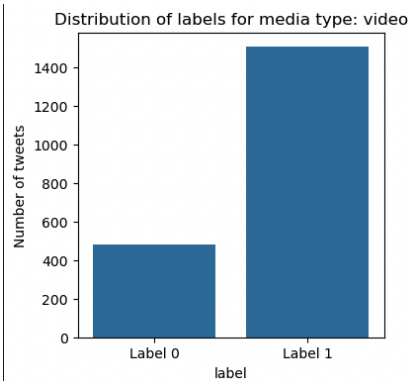


(d) Values of profile_background_tile per label

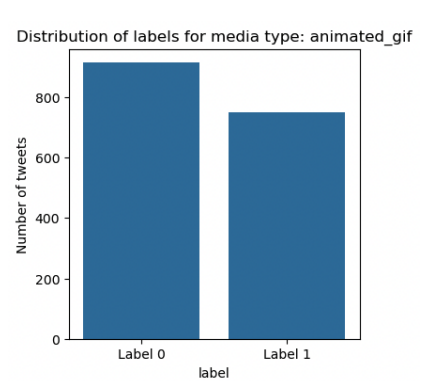
Figure 2: Binary features in the user field per label



(a) Tweet with a photo per label

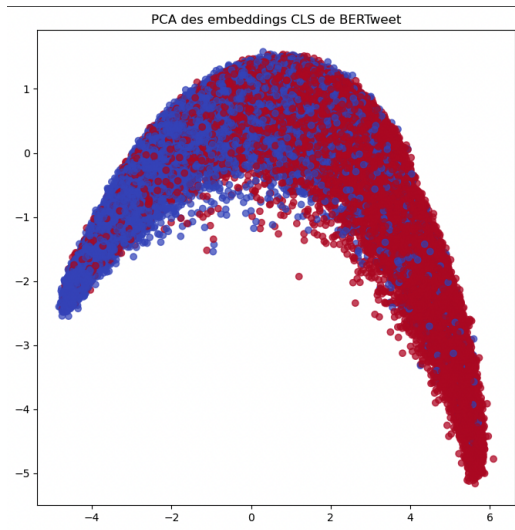


(b) Tweet with a video per label

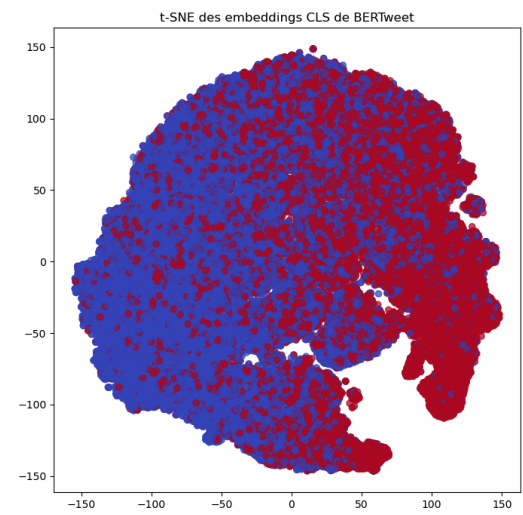


(c) Tweet with a gif per label

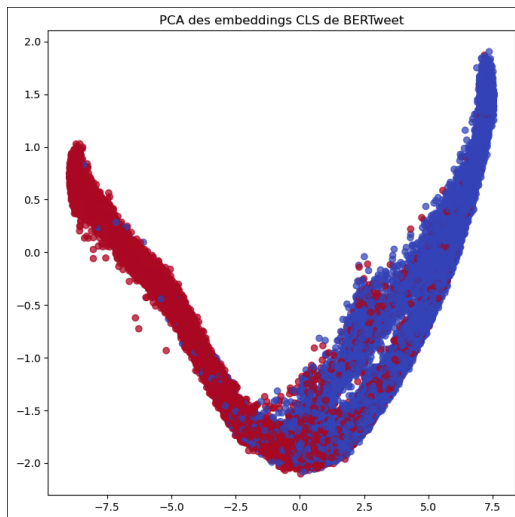
Figure 3: Distribution of entities.media per label



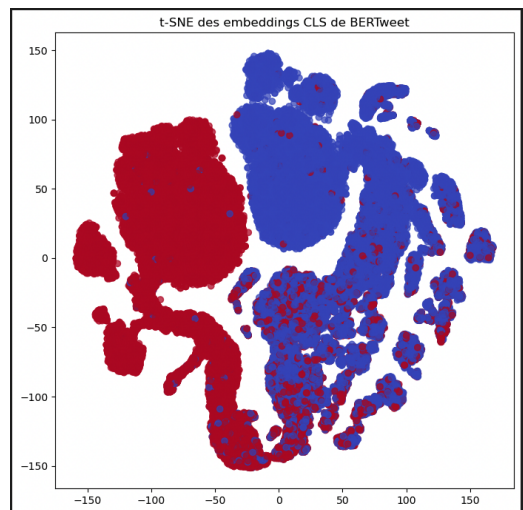
(a) PCA on full-text



(b) t-SNE on full-text



(c) PCA on user description



(d) t-SNE on user description

Figure 4: PCA projection and t-SNE obtained for the text fields with Bertweet

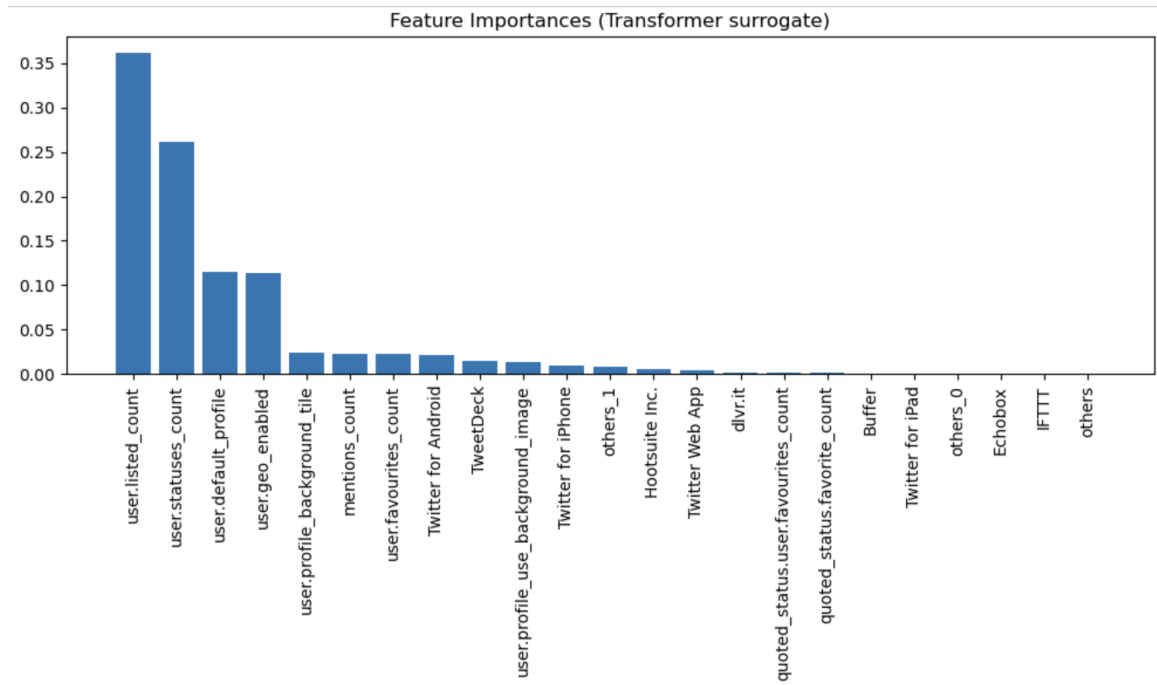


Figure 5: Features importance with IntegratedGradients

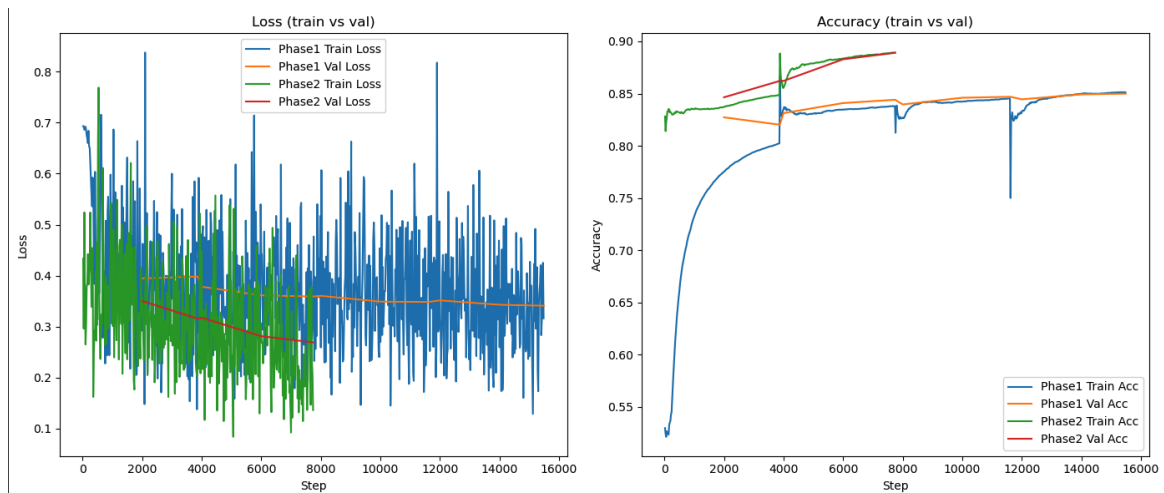


Figure 6: Overfitting in second phase : learning curve of the second transformer